CMSC131: Conditionals, Boolean Logic, and Switch

Chapter 3

By Naren Ayinala

October 2026

Reading order note — READ LATER: SWITCH AND TERNARY. The sections on the **switch statement** and the **ternary operator** are included here to keep conditional syntax together. **Skip both sections for now and return after learning methods (static and non-static) and constructors.** First focus on `if`, `else if`, `else`, Boolean operators, short-circuiting, and increment/decrement expressions.

Contents

1 Why Do We Need Conditionals? 3

2 Comparison Operators and Boolean Expressions 3

3 The Ordinary `if` Statement 4

3.1 One statement without braces . . . 4

3.2 The accidental semicolon . . . 4

4 Independent `if` Statements Versus a Chain 5

4.1 Several `if` statements back to back . . . 5

4.2 The `if / else if` chain . . . 5

4.3 An `if / else if` chain without `else` . . . 6

4.4 Why `else` cannot appear by itself . . . 6

4.5 The dangling `else` problem . . . 7

5 Boolean Logic: `&&`, `||`, and `!` 7

5.1 Short-circuit evaluation: why the right side may never run . . . 7

5.2 Short-circuiting can prevent an exception . . . 8

5.3 The right side might have a side effect . . . 8

5.4 Single `&` and `|` versus double `&&` and `||` . . . 9

5.5 Precedence and parentheses . . . 9

5.6 De Morgan's laws 9

6 Increment and Decrement: ++ and – 10

6.1 Prefix: update first, then use the new value 10

6.2 Postfix: use the old value, then update 10

6.3 Several worked traces 10

6.4 Short-circuiting and increments together 11

7 Nested Conditionals 11

8 The `switch` Statement — Read Later 12

8.1 Traditional `switch` with `case`, `break`, and `default` 12

8.2 Fall-through: what happens without `break`? 12

8.3 Strings and `switch` 13

8.4 Modern arrow switch labels 13

8.5 When to use `if` and when to use `switch` 14

9 Ternary Conditional Operator `?:` — Read Later 14

9.1 Ternary step-by-step 14

9.2 Only the chosen ternary branch runs 15

9.3 Ternary in a method (return value) 15

9.4 Nested ternaries 15

10 Complete Comparison and Reference Tables 16

11 Worked Output Traces 16

11.1 Trace 1: independent versus chained conditions 16

11.2 Trace 2: the right side is not always evaluated 16

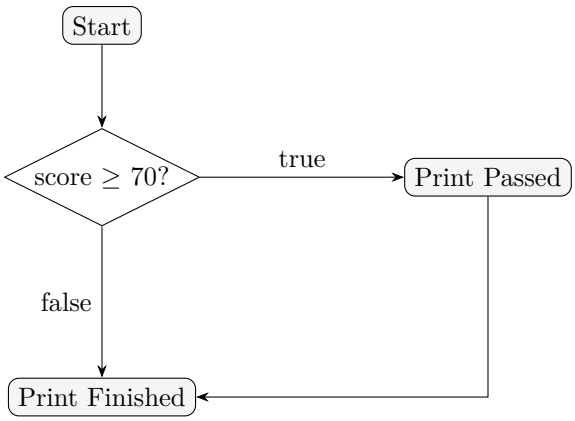
11.3 Trace 3: grouped switch cases 16

1 Why Do We Need Conditionals?

A Java program normally executes its statements in order, from top to bottom. But real programs often need to *make decisions*. For example, a student can pass or fail based on a score, and an ATM should allow a withdrawal only when enough money is available. A **conditional** lets the program execute code only when a condition is true. Every `if` condition must be a `boolean` expression: an expression whose value is `true` or `false`.

```
int score = 82;
if (score >= 70) {
    System.out.println("Passed!");
}
System.out.println("Finished checking.");
// Passed!
// Finished checking.
```

The first print executes only if `score >= 70` is true. The second print is outside the conditional and always executes once control reaches it. Notice that the braces `{` and `}` mark the group of statements controlled by the condition.



2 Comparison Operators and Boolean Expressions

A comparison checks a relationship between values. Java uses `==` for equality and `!=` for inequality. Remember that a single `=` is *assignment*, not a comparison.

Operator	Meaning	Example (when `x=5`)
`==`	Equal to	`x == 5` is true
`!=`	Not equal to	`x != 5` is false
`>`	Greater than	`x > 3` is true
`>=`	Greater than or equal	`x >= 5` is true
`<`	Less than	`x < 5` is false
`<=`	Less than or equal	`x <= 4` is false

```
int age = 18;
boolean adult = age >= 18; // true
if (adult) {
```

```
    System.out.println("Adult");
}
// if (age = 18) { } // ERROR: int assignment is not boolean
```

For Strings, use `.equals()` when comparing the text. `==` on object references asks whether they refer to the exact same object, not whether their contents match.

```
String password = new String("hello");
if (password.equals("hello")) {
    System.out.println("Correct password");
}
```

3 The Ordinary `if` Statement

3.1 One statement without braces

Java allows an `if` without braces, but then **only the immediately following statement** belongs to the `if`. Indentation does not change this rule.

```
int x = 2;
if (x > 5)
    System.out.println("Large");
System.out.println("Done"); // Always prints
// Output: Done
```

Compare this misleading indentation:

```
int x = 2;
if (x > 5)
    System.out.println("Large");
    System.out.println("Done"); // NOT part of the if
```

Even though the second print looks indented, Java controls only the first print. Use braces when learning Java and in almost all real code, because braces make the grouping explicit and prevent errors when adding new statements.

```
if (x > 5) {
    System.out.println("Large");
    System.out.println("Both lines run only when x > 5");
}
```

3.2 The accidental semicolon

A semicolon immediately after the condition is an *empty statement*. This means the `if` controls an empty operation, and the following block is separate.

```
int x = 1;
if (x > 10); { // The semicolon ends the if!
    System.out.println("Printed anyway");
}
```

4 Independent `if` Statements Versus a Chain

4.1 Several `if` statements back to back

Every independent `if` has its own test. Java evaluates them in order, even if an earlier one was true. **More than one block can run.**

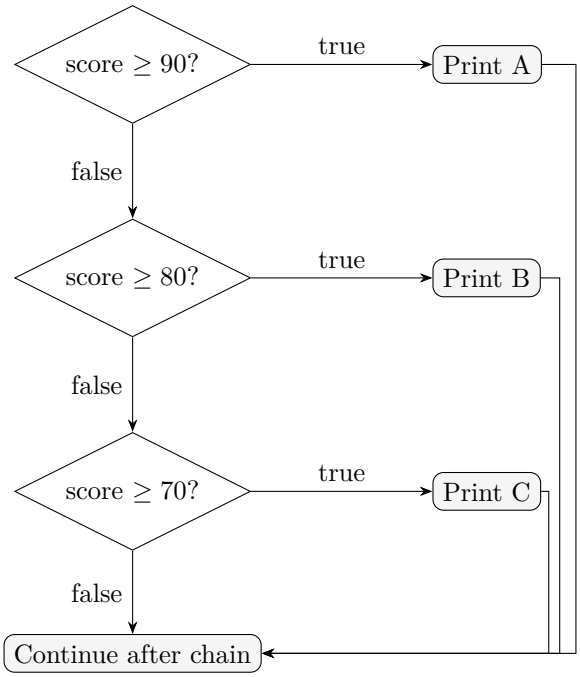
```
int score = 95;
if (score >= 60) System.out.println("Passed");
if (score >= 80) System.out.println("Good");
if (score >= 90) System.out.println("Excellent");
// Passed
// Good
// Excellent
```

All three conditions are true, so all three messages print. This is useful when several independent facts need checking.

4.2 The `if / else if` chain

An `else if` belongs to the same decision chain. Java checks the conditions from top to bottom, and as soon as one is true, it executes that branch and **skips all remaining branches**. At most one branch runs.

```
int score = 95;
if (score >= 90) {
    System.out.println("A");
} else if (score >= 80) {
    System.out.println("B");
} else if (score >= 70) {
    System.out.println("C");
}
// Output: A (not A, B, and C)
```



4.3 An `if / else if` chain without `else`

You do **not** need a final `else`. If every condition is false, no branch runs, and execution continues after the chain.

```
int temperature = 15;
if (temperature > 35) {
    System.out.println("Very hot");
} else if (temperature > 25) {
    System.out.println("Warm");
}
System.out.println("Done"); // Only this prints
```

A final `else` is useful when you want a fallback for *all remaining cases*. But it is not mandatory.

4.4 Why `else` cannot appear by itself

`else` means "otherwise" and must attach to a preceding unmatched `if` in the same statement structure. You cannot begin with `else`, and you cannot write an `else` after a completed, unrelated statement.

```
// else { System.out.println("No"); } // ERROR
if (true) {
    System.out.println("Yes");
} else {
    System.out.println("No");
}
```

A chain can have *zero or one* final `else`; it cannot have two final `else` branches. Also, `else if` is actually an `else` whose controlled statement is another `if`.

4.5 The dangling `else` problem

When braces are missing, `else` attaches to the **nearest preceding unmatched `if`**, regardless of indentation.

```
int a = 1, b = 0;
if (a > 0)
    if (b > 0)
        System.out.println("Both positive");
    else
        System.out.println("b is not positive");
// Prints: b is not positive
```

The `else` belongs to `if (b > 0)`, not `if (a > 0)`. Braces remove ambiguity.

Form	When branches run	If every condition is false
Independent `if`s	Zero, one, or several can run	None runs
`if / else if`	At most one branch runs	None runs
`if / else if / else`	Exactly one branch runs	Final `else` runs

5 Boolean Logic: `&&`, `||`, and `!`

A condition can combine multiple smaller Boolean expressions. `&&` means logical AND: both sides must be true. `||` means logical OR: at least one side must be true. `!` means NOT: it reverses a Boolean value.

```
int age = 20;
boolean hasTicket = true;
if (age >= 18 && hasTicket) {
    System.out.println("Allowed in");
}
if (age < 12 || age >= 65) {
    System.out.println("Discount");
}
boolean blocked = !hasTicket; // false
```

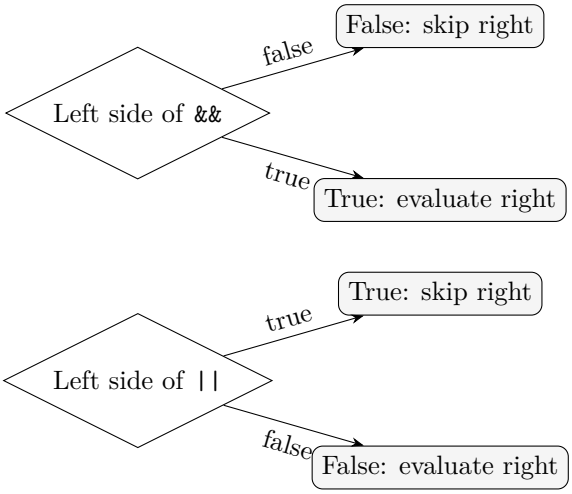
A	B	`A && B`	`A || B`	`!A`
false	false	false	false	true
false	true	false	true	true
true	false	false	true	false
true	true	true	true	false

5.1 Short-circuit evaluation: why the right side may never run

Java evaluates the *left* operand of `&&` or `||` first. Sometimes that result already determines the entire answer, so Java **does not evaluate the right operand at all**. This is called **short-circuit evaluation**.

- With `A && B`: if `A` is false, the whole expression must be false. Java skips `B`.
- With `A || B`: if `A` is true, the whole expression must be true. Java skips `B`.

- Otherwise Java must evaluate `B` to determine the result.



5.2 Short-circuiting can prevent an exception

```
int denominator = 0;
if (denominator != 0 && 10 / denominator > 2) {
    System.out.println("Big quotient");
}
// Safe: right side is skipped because left is false.
```

If the first condition is false, Java never attempts the division. This also matters when checking references before calling methods:

```
String name = null;
if (name != null && name.length() > 3) {
    System.out.println("Long name");
}
// Safe: name.length() is skipped.
```

The order is important. `name.length() > 3 && name != null` is unsafe because Java tries to call `length()` first.

5.3 The right side might have a side effect

A **side effect** means that evaluating an expression changes program state, such as incrementing a variable. Because short-circuiting may skip the right side, a variable might *not* change.

```
int count = 0;
boolean answer = false && (++count > 0);
System.out.println(answer); // false
System.out.println(count); // 0: ++count never ran
```

```
int count = 0;
boolean answer = true || (++count > 0);
System.out.println(answer); // true
System.out.println(count); // 0: ++count never ran
```

Compare the cases where the right side *must* run:

```
int a = 0;
boolean first = true && (++a > 0);
System.out.println(a); // 1
System.out.println(first); // true

int b = 0;
boolean second = false || (++b > 0);
System.out.println(b); // 1
System.out.println(second); // true
```

5.4 Single `&` and `|` versus double `&&` and `||`

For *Boolean operands*, Java also supports single `&` (AND) and single `|` (OR). Unlike the double operators, these **evaluate both operands every time**. Single `&` and `|` also have bitwise meanings for integral types; the examples below use Booleans only.

```
int x = 0;
boolean a = false & (++x > 0);
System.out.println(x); // 1: right side evaluated

int y = 0;
boolean b = true | (++y > 0);
System.out.println(y); // 1: right side evaluated
```

Operator	Result	Evaluate right side?
`&&`	AND	Only if left is true
`||`	OR	Only if left is false
`&`	Boolean AND	Always
`|`	Boolean OR	Always

5.5 Precedence and parentheses

`!` binds more tightly than `&&`, which binds more tightly than `||`. This means `a || b && c` is interpreted as `a || (b && c)`. Parentheses are recommended when mixing these operators.

```
boolean a = true, b = false, c = false;
System.out.println(a || b && c); // true
System.out.println((a || b) && c); // false
```

5.6 De Morgan's laws

When negating a combined condition, reverse the operator and negate each part:

$$\neg(A \wedge B) = (\neg A) \vee (\neg B), \qquad \neg(A \vee B) = (\neg A) \wedge (\neg B).$$

```
boolean a = true, b = false;
System.out.println(!(a && b)); // true
System.out.println(!a || !b); // true
System.out.println(!(a || b)); // false
System.out.println(!a && !b); // false
```

6 Increment and Decrement: ++ and –

The ++ operator adds one to a numeric variable; – subtracts one. They come in two forms: **prefix** (before the variable) and **postfix** (after the variable). If they appear as standalone statements, both forms update the variable the same way.

```
int x = 5;
x++; // x is 6
++x; // x is 7
x--; // x is 6
--x; // x is 5
```

6.1 Prefix: update first, then use the new value

```
int x = 5;
int y = ++x;
System.out.println(x); // 6
System.out.println(y); // 6
```

6.2 Postfix: use the old value, then update

```
int x = 5;
int y = x++;
System.out.println(x); // 6
System.out.println(y); // 5
```

Expression	Value used	Variable afterward	Example
`++x`	new value	increased by 1	$x = 5$: value 6, $x = 6$
`x++`	old value	increased by 1	$x = 5$: value 5, $x = 6$
`-x`	new value	decreased by 1	$x = 5$: value 4, $x = 4$
`x-`	old value	decreased by 1	$x = 5$: value 5, $x = 4$

6.3 Several worked traces

```
int a = 3;
System.out.println(a++); // 3
System.out.println(a); // 4
System.out.println(++a); // 5
System.out.println(a); // 5
```

```
int x = 2;
int y = x++ + 5;
// x++ contributes 2, then x becomes 3.
// y = 2 + 5 = 7
System.out.println(x + ", " + y); // 3, 7
```

```
int x = 2;
int y = ++x + 5;
// ++x changes x to 3 before contributing 3.
// y = 3 + 5 = 8
System.out.println(x + ", " + y); // 3, 8
```

```
int n = 1;
int result = n++ + ++n;
// Left operand contributes 1; n becomes 2.
// Right operand increments n to 3 and contributes 3.
// result = 1 + 3 = 4; n = 3
System.out.println(result + ", " + n); // 4, 3
```

Java evaluates operands left to right. Complex expressions with multiple increments are legal but difficult to read; use separate statements in real programs.

6.4 Short-circuiting and increments together

```
int x = 0;
if (x++ > 0 && ++x > 1) {
    System.out.println("Inside");
}
System.out.println(x); // 1
```

Step 1: `x++ > 0` compares the *old* value 0 with 0, so the left side is false. Step 2: `x` increases to 1. Step 3: because the left side of `&&` is false, Java skips `++x > 1`. The final value is 1, not 2.

```
int x = 0;
if (x++ == 0 || ++x > 1) {
    System.out.println("Inside");
}
System.out.println(x); // 1
```

Here the left side of `||` is true (the old value is 0), so Java skips the right side. The block prints `Inside`, and `x` is 1.

7 Nested Conditionals

An `if` can appear inside another `if`. The inner condition is checked only when execution enters the outer block.

```
int age = 19;
boolean hasID = false;
if (age >= 18) {
    if (hasID) {
```

```
        System.out.println("Enter");
    } else {
        System.out.println("Bring your ID");
    }
} else {
    System.out.println("Too young");
}
// Bring your ID
```

For this example, `if (age >= 18 && hasID)` can combine the two positive checks, but nested conditionals are useful when different failure cases require different messages.

8 The `switch` Statement — Read Later

> **READ LATER:** Skip this section until after learning static and non-static methods and constructors. It is included here as a reference alongside the ternary operator.

A `switch` selects behavior based on one expression. It is useful when you have many discrete values such as menu choices, day numbers, characters, Strings, or enum constants. In an introductory course, the traditional `switch` statement is a good starting point.

8.1 Traditional `switch` with `case`, `break`, and `default`

```
int day = 2;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    default:
        System.out.println("Unknown day");
}
// Tuesday
```

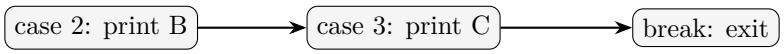
Java evaluates `day` once, finds the matching `case`, and begins executing from there. `break` exits the switch. The optional `default` label handles values that do not match any case. The order of `case` labels does not have to be numerical.

8.2 Fall-through: what happens without `break`?

In the traditional colon form, execution continues into subsequent cases until a `break`, `return`, another transfer of control, or the end of the switch. It does *not* re-check each next case label.

```
int choice = 2;
switch (choice) {
    case 1: System.out.print("A");
```

```
    case 2: System.out.print("B");
    case 3: System.out.print("C"); break;
    default: System.out.print("D");
}
// Output: BC
```



Sometimes fall-through is intentional, for example grouping several case labels together:

```
char grade = 'B';
switch (grade) {
    case 'A':
    case 'B':
    case 'C':
        System.out.println("Passing grade");
        break;
    default:
        System.out.println("Other grade");
}
```

8.3 Strings and `switch`

Java supports Strings as switch selectors. String cases compare their textual values, rather than requiring the same object reference.

```
String command = "start";
switch (command) {
    case "start":
        System.out.println("Running");
        break;
    case "stop":
        System.out.println("Stopped");
        break;
    default:
        System.out.println("Unknown command");
}
```

A traditional switch accepts selectors such as `byte`, `short`, `char`, `int`, their wrapper types, `String`, and enums. It does not directly accept `boolean`, `long`, `float`, or `double` in the standard introductory forms. A null selector causes a `NullPointerException` in the traditional forms shown here.

8.4 Modern arrow switch labels

Modern Java also supports `case ... ->` labels. They do not fall through, so they are often easier to read.

```
int day = 2;
switch (day) {
    case 1 -> System.out.println("Monday");
    case 2 -> System.out.println("Tuesday");
    default -> System.out.println("Other");
}
```

This syntax requires a sufficiently recent Java version (standard since Java 14). Use the syntax expected by your course or assignment.

8.5 When to use `if` and when to use `switch`

Use `if` when conditions involve ranges, multiple Boolean tests, or unrelated comparisons. Use `switch` when comparing one selector against many discrete choices. For example, `score >= 90` fits `if`, whereas matching a menu choice of 1, 2, or 3 fits `switch`.

9 Ternary Conditional Operator `?:` — Read Later

> **Important learning-order note:** Read this section **after you have learned how to write and call methods, including static and non-static methods, and after you understand constructors**. The operator itself is simple, but later examples will make more sense when you already understand expressions, return values, and methods. It is included here so that all conditional syntax stays in one document.

The ternary operator is an **expression** that chooses between two values. Its syntax is

```
condition ?  expressionIfTrue :  expressionIfFalse
```

First Java evaluates the Boolean condition. If true, it evaluates only the expression after `?`. If false, it evaluates only the expression after `:`. The other branch is **not evaluated**. The selected result becomes the value of the whole expression.

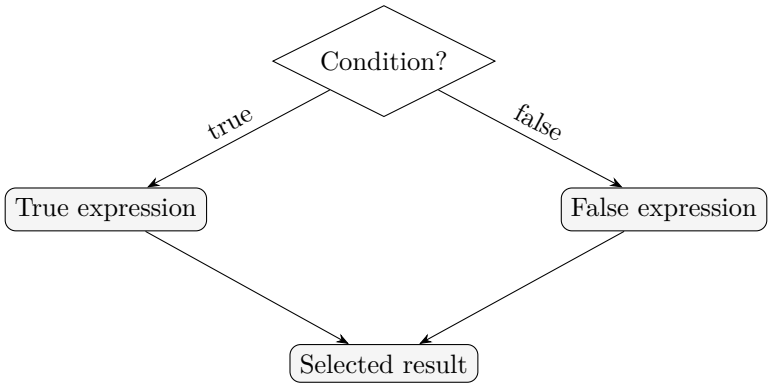
```
int age = 20;
String label = (age >= 18) ? "adult" : "minor";
System.out.println(label); // adult
```

This is equivalent in effect to assigning the variable in an ordinary `if/else`:

```
int age = 20;
String label;
if (age >= 18) {
    label = "adult";
} else {
    label = "minor";
}
```

9.1 Ternary step-by-step

```
int x = 7;
int answer = (x % 2 == 0) ? 100 : 200;
// 7 % 2 is 1; 1 == 0 is false.
// Only 200 is chosen.
System.out.println(answer); // 200
```



9.2 Only the chosen ternary branch runs

Just like short-circuiting, the unchosen expression is skipped, which can matter if an expression has a side effect.

```
int x = 0;
int y = (true) ? ++x : ++x;
System.out.println(x); // 1, not 2
System.out.println(y); // 1
```

```
int denominator = 0;
int answer = (denominator == 0) ? 0 : 10 / denominator;
System.out.println(answer); // 0; division is skipped
```

9.3 Ternary in a method (return value)

Once you understand methods, a ternary expression can be useful for a short returned value:

```
public static String describe(int score) {
    return (score >= 70) ? "pass" : "fail";
}
```

The method is `static` here only to give a compact example. The ternary operator works equally well in instance methods; it is not tied to static methods.

9.4 Nested ternaries

Nested ternaries are legal, but harder to read. Parentheses make the grouping clear:

```
int score = 85;
String grade = (score >= 90) ? "A"
             : (score >= 80) ? "B"
             : (score >= 70) ? "C" : "D";
System.out.println(grade); // B
```

For several branches, prefer an `if / else if / else` chain. The ternary is best for choosing between two simple expressions. Both alternatives must be compatible with the context in which the expression is used.

10 Complete Comparison and Reference Tables

Construct	How it works	Typical use
`if`	Runs block if condition true	Optional action
Several `if`s	Each condition evaluated independently	Multiple independent actions
`if / else if`	First true branch only; possibly none	Prioritized choices
`if / else`	Exactly one of two branches	Either/or decision
`if / else if / else`	Exactly one branch	Multiple choices with fallback
`&&`	AND, short-circuit if left false	Require both conditions
`||`	OR, short-circuit if left true	Require either condition
`!`	Negates Boolean value	Reverse condition
`switch`	Select matching case	Discrete options
`?:`	Select one of two expression values	Compact conditional expression

11 Worked Output Traces

11.1 Trace 1: independent versus chained conditions

```
int x = 10;
if (x > 3) System.out.print("A");
if (x > 8) System.out.print("B");
if (x > 12) System.out.print("C");
// AB
```

```
int x = 10;
if (x > 3) System.out.print("A");
else if (x > 8) System.out.print("B");
else if (x > 12) System.out.print("C");
// A
```

11.2 Trace 2: the right side is not always evaluated

```
int x = 2;
boolean test = (x++ > 2) || (++x > 3);
// First: old x=2, comparison false, x becomes 3.
// Second: ++x makes x=4, comparison true.
System.out.println(x); // 4
System.out.println(test); // true
```

11.3 Trace 3: grouped switch cases

```
int n = 3;
switch (n) {
    case 1:
    case 2:
        System.out.print("Small"); break;
```

```
    case 3:
    case 4:
        System.out.print("Medium"); break;
    default:
        System.out.print("Other");
}
// Medium
```